Zahlensysteme

- Dezimal $10^3\ 10^2\ 10^1\ 10^0$
  Binär $2^3\ 2^2\ 2^1\ 2^0$
- Hexadezimal 1, 2, ..., 9, A, B, C, D, E, F : $16^3\ 16^2\ 16^1\ 16^0$ → 4 Bits umrechnung
- Oktalsystem 1, 2, ..., 7, 10, 11, ..., 17, 21... $8^3\ 8^2\ 8^1\ 8^0$ → 3 Bit-umrechnung

Aussagenlogik

- wahr oder falsch
- nicht p → ¬p
- p und q → p ∧ q
- p oder q → p ∨ q
- entweder p oder q → p xor q
- wenn p wahr ist ist q auch wahr: p → q
- äquivalent, ist p wahr/falsch ist q auch wahr/falsch: p ↔ q

p	q	p ∧ q
w	w	w
w	f	f
f	w	f
f	f	f

p	q	p xor q
w	w	f
w	f	w
f	w	w
f	f	f

p	¬p
w	f
f	w

p	q	p ∨ q
w	w	w
w	f	w
f	w	w
f	f	f

p	q	p → q
w	w	w
w	f	f
f	w	w
f	f	w

Bitmaske

- um bestimmte Bits zu lesen bzw. zu setzen
- Lesen: ∧ 1
  Setzen: ∨ 1
  Löschen: ∧ 0
  Umschalten xor 1

Zahlen mit Vorzeichen

- 0 → + 1 → - 1. Zahl als Vorzeichen
- Einerkomplement: normale Darstellung für positive Zahlen
  negative Zahlen → umgedreht für 1 → 0
- Zweierkomplement: positiv → normale Darstellung
  negativ → zu Einerkomplement + 1 dazu

Subtraktion möglich

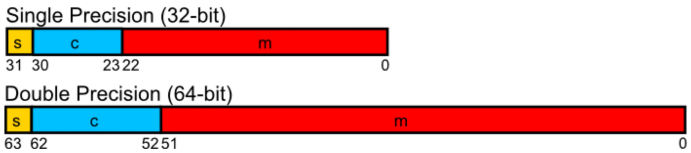
Festkommazahlen

- Vorzeichenbit S, Ganzzahliger I-Bit, Gebrochener F-Bit
  BSP I = 4 F = 3 für (0) 0 1 0 0 1 0 1
  $= 1 \cdot 2^2 + 1 \cdot 2^{-2} + 1 \cdot 2^{-4} = 4{,}625$
- Vorgehen zu Berechnung: dezimal zu Binär: BSP.

	0,625	0
×2	1,250	0,1
	0,250	
×2	0,5	0,10
	0,5	
×2	1,0	0,101
	0	

Gleitkommazahlen

- Vorzeichen V, Exponent E, Mantisse M $\rightarrow$ $Z = (-1)^V \cdot M \cdot 2^E$



- Vorgehen:
  1. Berechnung Bias $2^{r-1}-1$ mit r Anzahl Bits von c
  2. Umrechnung in Festkommazahl (ohne Vorzeichen)
  3. Normieren
  4. Berechnung Exponent (Bias berücksichtigen)
  5. Vorzeichenbit bestimmen
  6. Zusammensetzen

Bsp: 18,4 single

1) Bias $2^{8-1}-1 = 127$

2) 18 $\rightarrow$ 1 0 0 1 0

0,4 $\rightarrow$

x2	0,4	0,
x2	0,8	0,0
	1,6	0,01
x2	0,6	
	1,2	0,011
x2	0,2	
	0,4	$0,0\overline{11}$

$\rightarrow 10010,\overline{011}$

3) $1,0010011 \cdot 2^4$

4) $E = 127 + 4 = 131$

5) $V = (-1)^0 = 1 \rightarrow 0$

6) $\underbrace{0}_{V}\ \underbrace{10000011}_{E}\ \underbrace{00100110110110110}_{M}$

Programmiervorgehen

- imperativ $\rightarrow$ befehls-, problemorientiert, prozedural
- deklarativ / funktional $\rightarrow$ logisch

Aufzählung

Bsp: enum Monat {Januar, Februar, März, April};
Monat aktueller_Monat = Januar;

- weist automatisch Werte zu, mit 0 beginnend
  Start kann verändert werden: enum Monat {Januar = 10, Februar ...}

Strukturen

- Zusammenfassung von Datentypen

```
Bsp  struct adresse {
        char vorname [20];
        int plz;
     }
adresse meine_adresse;
meine_adresse.vorname = "Max";
adresse zweitwohnsitz ={"Max", "73728"}
```

- array von struct

```
Bsp: adresse adressbuch[10],
     adressbuch [0] = zweitwohnsitz;
     adressbuch [1]. name = "Karl";
```

Bitfelder

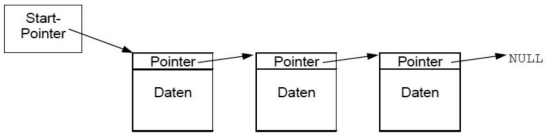
- effektive Speichernutzung -> nur benötigte Anzahl an Bits wird reserviert
  Bsp: int hour : 4;

Dynamische Datenstrukturen

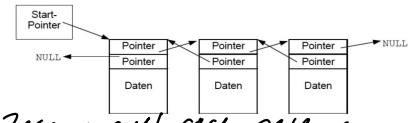
- Arrays oft mit festgelegtem Speicherplatz -> grenzt ein oder verbraucht unnötig Speicherplatz
- Listen, Bäume, Graphen

Verkettete Listen

- einfach verkettete Liste -> Zeiger auf Folgeobjekt



- doppelt verkettete Liste -> Zeiger auf Folge- und Vorgängerobjekt



- Struct um Zeiger pnext und pprevious erweitert -> Zeiger auf sich selbst